

Tugas Praktikum 8
Sistem Informasi Geografis

Nama : Awi Septian Prasetyo
NIM : 123140201
Kelas : Sistem Informasi Geografis
Dosen : Muhammad Habib Alghifari, S.kom., M.T.I.
Alya Khairunnisa Rizkita, S.Kom., M.Kom.
Github : https://github.com/Awesome1209/sig_123140201.git

Deskripsi Tugas

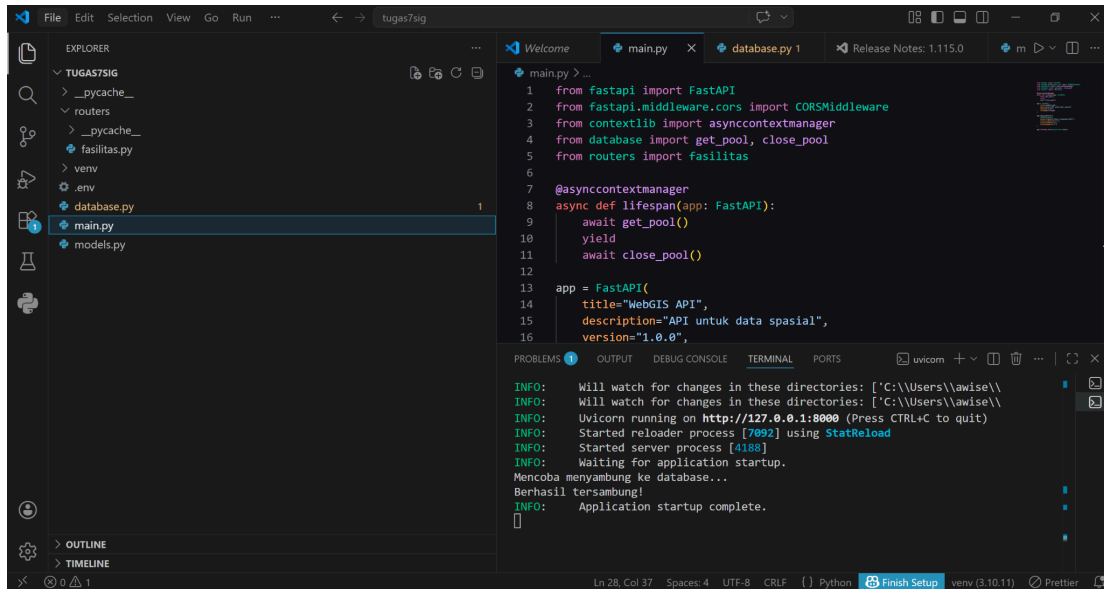
Bangun aplikasi WebGIS frontend dengan React dan Leaflet yang terintegrasi dengan API FastAPI dari tugas sebelumnya.

Ketentuan

- Tampilkan peta dengan center di wilayah data Anda
- Fetch dan tampilkan data GeoJSON dari API
- Implementasi popup dengan informasi lengkap
- Styling berbeda untuk setiap jenis/kategori data
- Tambahkan minimal satu interaksi (hover highlight atau zoom)

Langkah-langkah Pengerjaan:

1. Menyiapkan Backend FastAPI



Gambar 1. Tampilan Pengerjaan Backend

Langkah pertama adalah menyiapkan backend FastAPI yang berfungsi sebagai penyedia data untuk frontend. Backend ini terdiri dari beberapa file utama, yaitu: main.py sebagai file utama aplikasi FastAPI, database.py sebagai koneksi database PostgreSQL, models.py untuk mendefinisikan schema input, dan routers/fasilitas.py untuk menangani endpoint API fasilitas.

Pada main.py, aplikasi FastAPI dibuat dan router fasilitas didaftarkan ke aplikasi. Pada database.py, koneksi ke database dibuat menggunakan asyncpg dengan create_pool, sehingga backend dapat berkomunikasi dengan database PostgreSQL/PostGIS. Pada models.py, schema FasilitasCreate dibuat untuk menerima input berupa nama, jenis, alamat, longitude, dan latitude. Selain itu juga terdapat FasilitasResponse untuk struktur keluaran data fasilitas.

- Pada routers/fasilitas.py, saya menggunakan beberapa endpoint penting:
- GET /api/fasilitas/
- GET /api/fasilitas/geojson
- GET /api/fasilitas/nearby
- GET /api/fasilitas/{id}
- POST /api/fasilitas/

Endpoint terpenting untuk frontend adalah GET /api/fasilitas/geojson, karena endpoint ini mengubah data geometri dari database menjadi format FeatureCollection, lalu mengirim properti id, nama, jenis, dan alamat ke frontend. Endpoint POST /api/fasilitas/ digunakan untuk memasukkan data baru, sekaligus membuat tabel fasilitas jika belum ada.

2. Menjalankan Backend

```
(venv) PS C:\Users\awise\Downloads> cd tugas7sig
INFO: Will watch for changes in these directories: ['C:\\Users\\awise\\
INFO: Will watch for changes in these directories: ['C:\\Users\\awise\\
Down: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Will watch for changes in these directories: ['C:\\Users\\awise\\Downloads\\tugas7sig']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [7092] using StatReload
INFO: Started server process [4188]
INFO: Waiting for application startup.
Mencoba menyambung ke database...
Berhasil tersambung!
```

Gambar 2. Backend FastAPI berhasil dijalankan melalui terminal

Setelah file backend selesai, saya menjalankan backend menggunakan terminal dengan perintah:

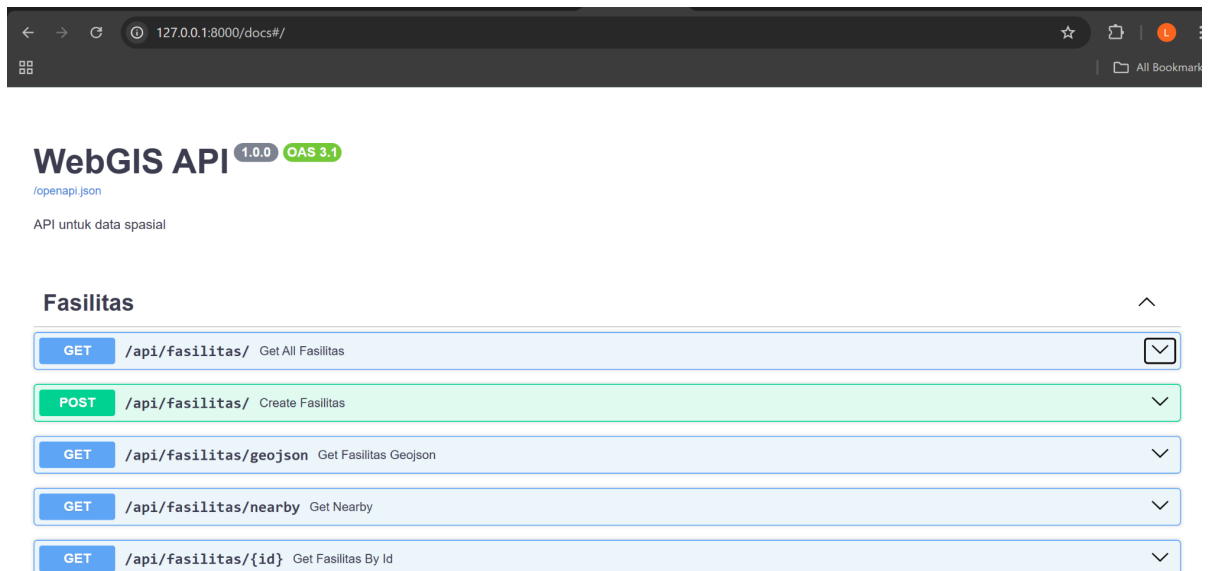
```
uvicorn main:app --reload
```

Setelah backend berhasil berjalan, API dapat diakses melalui:

```
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/api/fasilitas/geojson
```

Halaman /docs digunakan untuk mencoba endpoint secara langsung melalui Swagger UI, sedangkan endpoint /api/fasilitas/geojson digunakan oleh frontend untuk mengambil data spasial.

3. Memasukkan Data ke Backend



Gambar 3. Proses input data lokasi melalui Swagger UI

Data lokasi dimasukkan melalui Swagger UI menggunakan endpoint POST /api/fasilitas/. Setiap lokasi diinput dengan format JSON yang berisi:

- nama

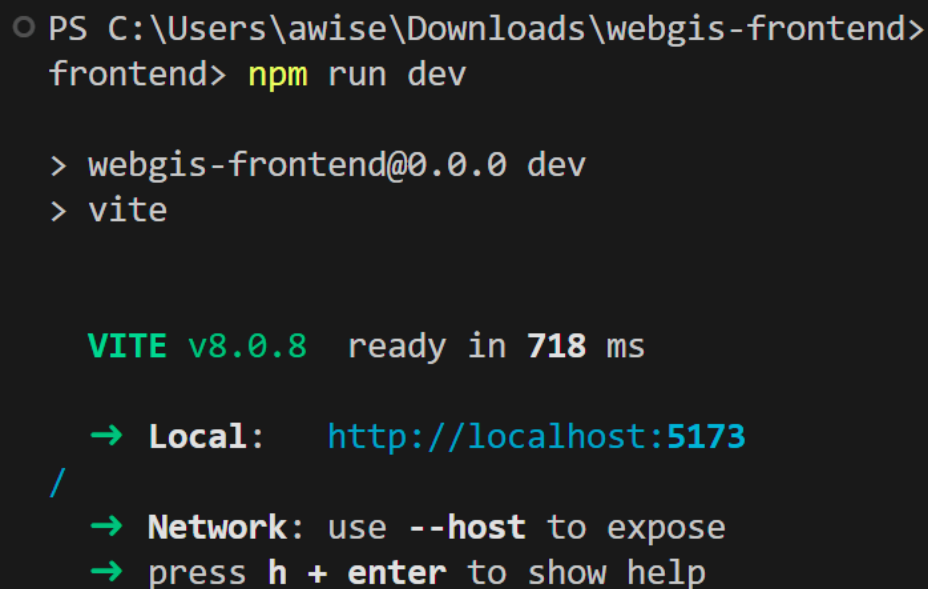
- jenis
- alamat
- longitude
- latitude

Salah satu contoh:

```
{
  "nama": "Pasar Way Halim",
  "jenis": "Pasar",
  "alamat": "Jl. Gunung Rajabasa Raya, No. T.21, Way Halim, 35361, Perumnas Way Halim, Kec. Way Halim, Kota
Bandar Lampung, Lampung 35132",
  "longitude": 105.274965,
  "latitude": -5.376959
}
```

Data dimasukkan satu per satu hingga semua 7 titik lokasi berhasil tersimpan di database. Setelah itu, saya memeriksa endpoint `/api/fasilitas/geojson` untuk memastikan bahwa data berhasil dikembalikan dalam format `FeatureCollection`. Endpoint ini memang dirancang untuk membaca data geometri dengan `ST_AsGeoJSON(geom)` lalu mengemasnya dalam `GeoJSON`.

4. Membuat Project Frontend React dengan Vite



```
PS C:\Users\awise\Downloads\webgis-frontend>
frontend> npm run dev

> webgis-frontend@0.0.0 dev
> vite

VITE v8.0.8 ready in 718 ms

➔ Local:   http://localhost:5173
/
➔ Network: use --host to expose
➔ press h + enter to show help
```

Gambar 4. Pembuatan project frontend React menggunakan Vite

Setelah backend siap, saya membuat project frontend baru menggunakan Vite dengan perintah:

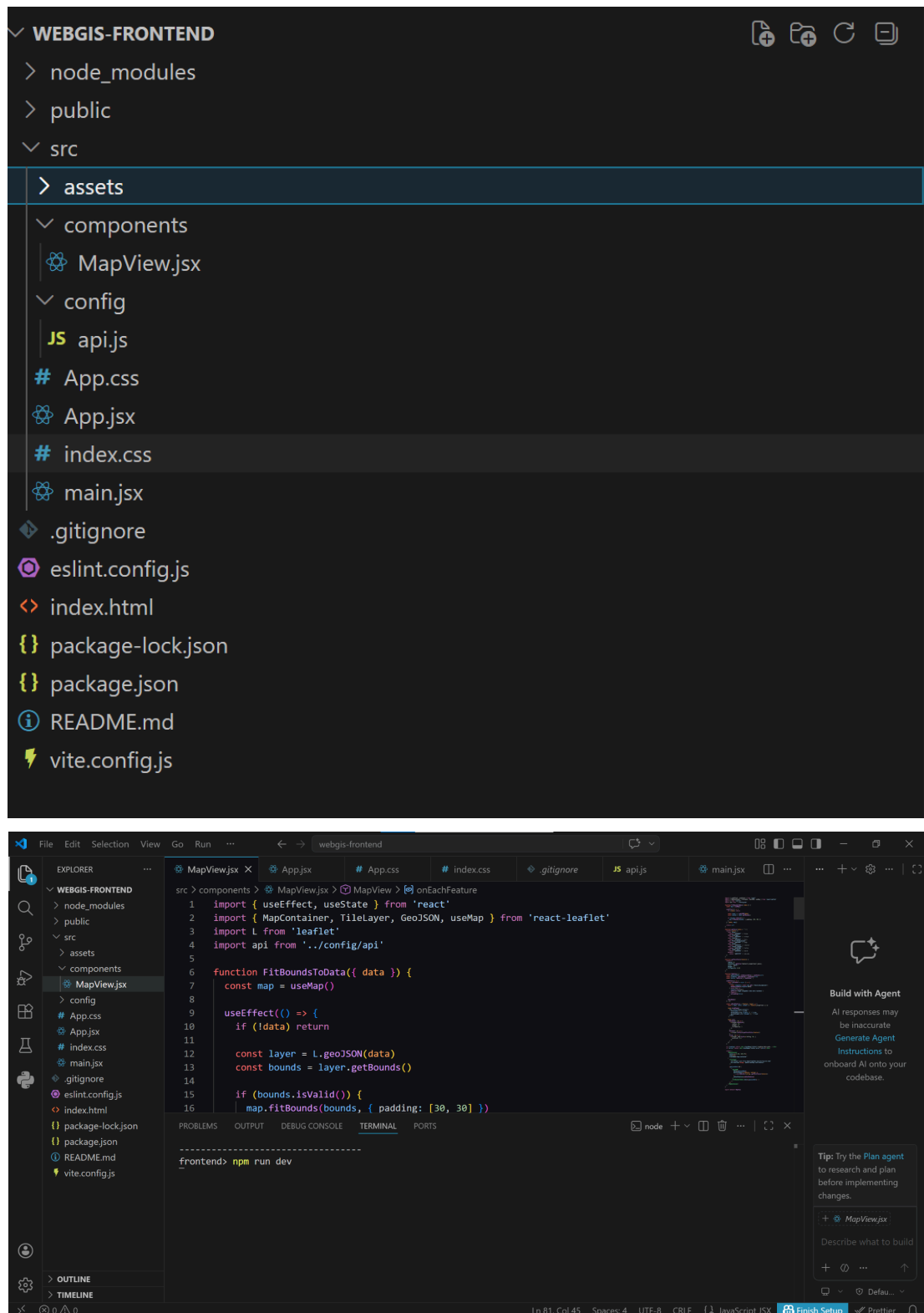
```
npm create vite@latest webgis-frontend
```

Pada saat setup project, pilih: React dan JavaScript. Setelah itu saya masuk ke folder project dan menginstal dependency yang diperlukan:

```
cd webgis-frontend
npm install
npm install leaflet react-leaflet axios
```

Materi praktikum memang menggunakan alur pembuatan project React dengan Vite dan instalasi leaflet, react-leaflet, serta axios.

5. Menyusun Struktur Frontend



Gambar 5. Struktur file frontend React

Pada folder frontend, saya membuat struktur file seperti berikut:

```
src/
├── components/
│   └── MapView.jsx
├── config/
│   └── api.js
├── App.jsx
├── App.css
├── index.css
└── main.jsx
```

Fungsi dari masing-masing file adalah:

- MapView.jsx untuk menampilkan peta dan data GeoJSON
- api.js untuk konfigurasi base URL Axios
- App.jsx sebagai komponen utama
- App.css dan index.css untuk styling tampilan
- main.jsx sebagai entry point React

6. Menghubungkan Frontend ke Backend

```
src > config > JS api.js > [🔗] default
1  import axios from 'axios'
2
3  const api = axios.create({
4    baseUrl: 'http://localhost:8000/api',
5    timeout: 10000,
6    headers: {
7      'Content-Type': 'application/json'
8    }
9  })
10
11 export default api
```

Gambar 6. Konfigurasi koneksi frontend ke backend menggunakan Axios

Pada file src/config/api.js, saya membuat konfigurasi Axios:

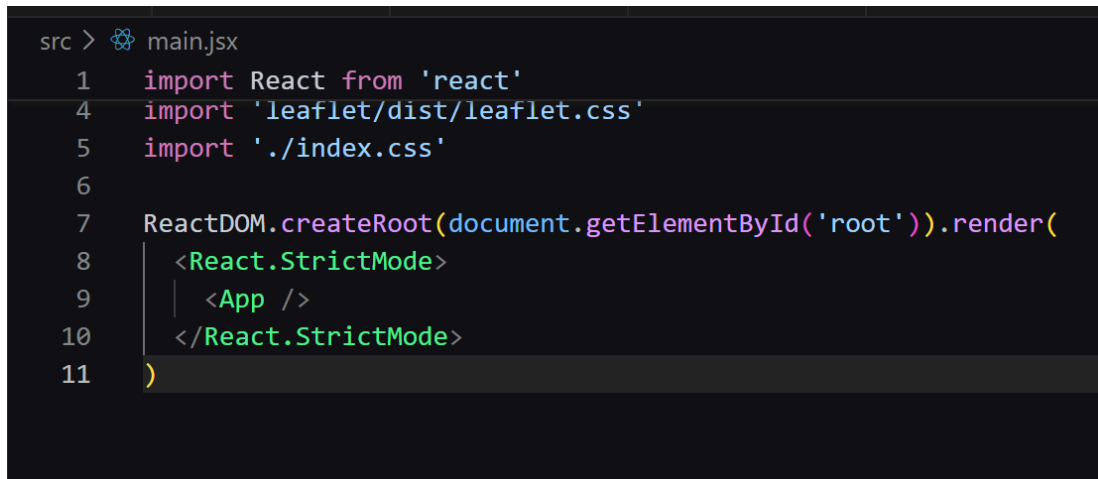
```
import axios from 'axios'

const api = axios.create({
  baseURL: 'http://localhost:8000/api',
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json'
  }
})

export default api
```

Konfigurasi ini memudahkan proses request dari frontend ke backend. Pada materi praktikum, pendekatan yang sama juga digunakan dengan membuat Axios instance agar base URL API dapat dikelola secara terpusat.

7. Mengimpor CSS Leaflet

A screenshot of a code editor with a dark background. The file path 'src > main.jsx' is shown at the top left. The code is as follows:

```
1  import React from 'react'
4  import 'leaflet/dist/leaflet.css'
5  import './index.css'
6
7  ReactDOM.createRoot(document.getElementById('root')).render(
8    <React.StrictMode>
9      <App />
10   </React.StrictMode>
11 )
```

Gambar 7. Import CSS Leaflet pada file main.jsx

Pada file src/main.jsx, saya menambahkan import CSS Leaflet:

```
import 'leaflet/dist/leaflet.css'
```

Langkah ini sangat penting karena tanpa CSS Leaflet, tampilan peta, tile, popup, dan kontrol peta tidak akan muncul dengan baik. Hal ini juga ditekankan secara langsung pada materi praktikum.

8. Membuat Komponen Peta

```
src > components > MapView.jsx > MapView > onEachFeature
44 function getPointStyle(feature) {
45   return {
46     radius: 8,
47     fillColor: getColor(feature?.properties?.jenis),
48     color: '#222',
49     weight: 1,
50     fillOpacity: 0.85
51   }
52 }
53
54 function MapView() {
55   const [geojsonData, setGeojsonData] = useState(null)
56   const [loading, setLoading] = useState(true)
57   const [error, setError] = useState('')
58
59   useEffect(() => {
60     const fetchData = async () => {
61       try {
62         const response = await api.get('/fasilitas/geojson')
63         setGeojsonData(response.data)
64       } catch (err) {
65         console.error(err)
66         setError('Gagal mengambil data dari backend.')
67       } finally {
68         setLoading(false)
69       }
60     }
61   })
62 }
```

Gambar 8. Pembuatan komponen peta utama pada MapView.jsx

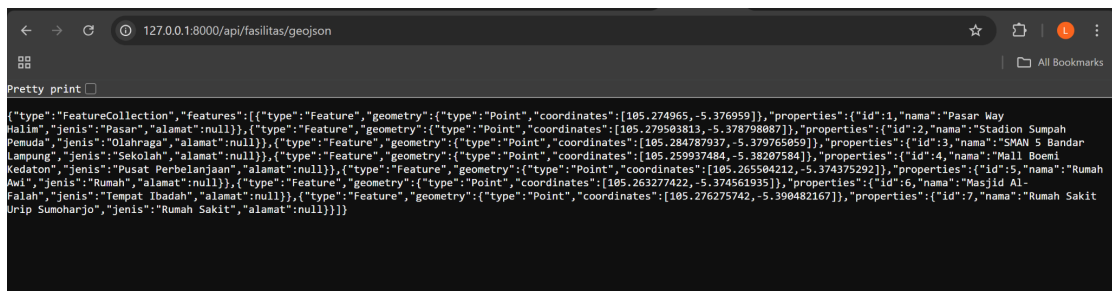
Pada file src/components/MapView.jsx, saya membuat komponen peta menggunakan MapContainer, TileLayer, GeoJSON, dan useMap.

Peta diatur memiliki center awal di sekitar wilayah data saya, yaitu:

- center={[-5.38, 105.27]}
- zoom={13}

Selain itu, saya menambahkan fungsi FitBoundsToData agar peta otomatis menyesuaikan area ketika data sudah berhasil dimuat.

9. Mengambil Data GeoJSON dari API



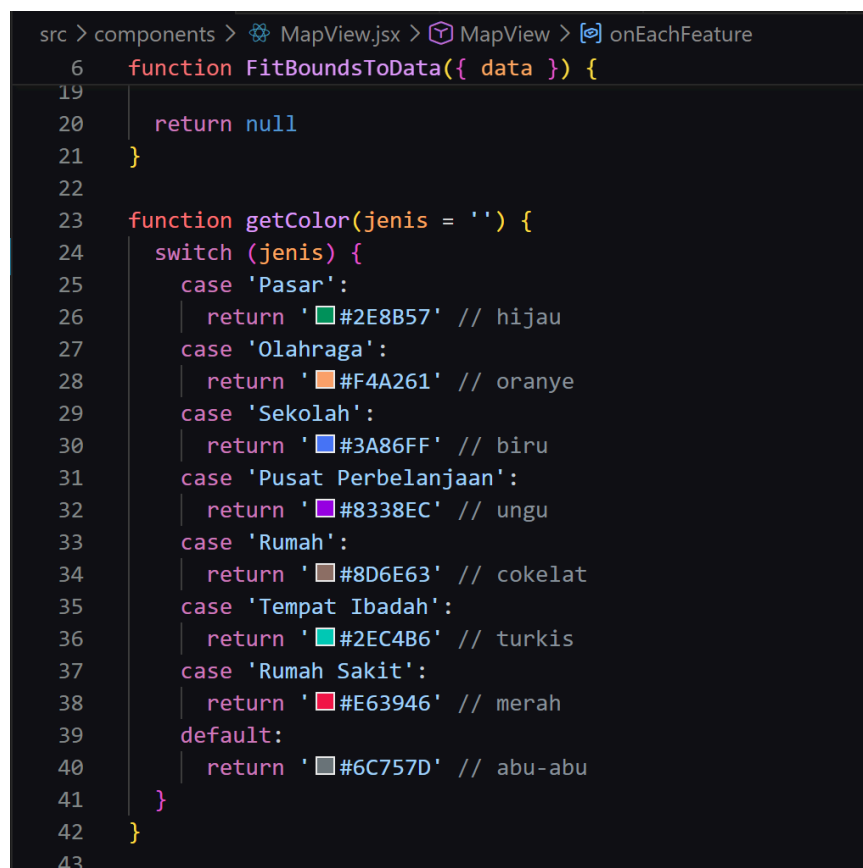
Gambar 9. Hasil endpoint /api/fasilitas/geojson

Data diambil dari backend menggunakan useEffect dan Axios:

```
const response = await api.get('/fasilitas/geojson')
setGeojsonData(response.data)
```

Saat halaman dibuka, frontend akan otomatis memanggil endpoint /fasilitas/geojson, lalu data yang diterima disimpan ke state geojsonData. Setelah itu data ditampilkan pada peta menggunakan komponen `<GeoJSON />`. Pendekatan ini sesuai dengan materi yang mencontohkan pengambilan data dari API menggunakan useEffect dan axios, lalu menampilkannya sebagai layer GeoJSON.

10. Memberikan Styling Berbeda pada Setiap Kategori



Gambar 10. Styling warna berbeda pada titik lokasi

Agar setiap kategori lokasi memiliki tampilan berbeda, saya membuat fungsi `getColor()` yang mengembalikan warna berbeda untuk setiap jenis data:

Pasar → hijau
Olahraga → oranye
Sekolah → biru
Pusat Perbelanjaan → ungu
Rumah → coklat
Tempat Ibadah → turkis
Rumah Sakit → merah

Kemudian warna ini diterapkan pada marker titik menggunakan `L.circleMarker` di dalam `pointToLayer`. Dengan demikian, setiap titik di peta memiliki warna yang sesuai dengan kategorinya. Ini memenuhi ketentuan tugas yang meminta styling berbeda untuk setiap jenis atau kategori data.

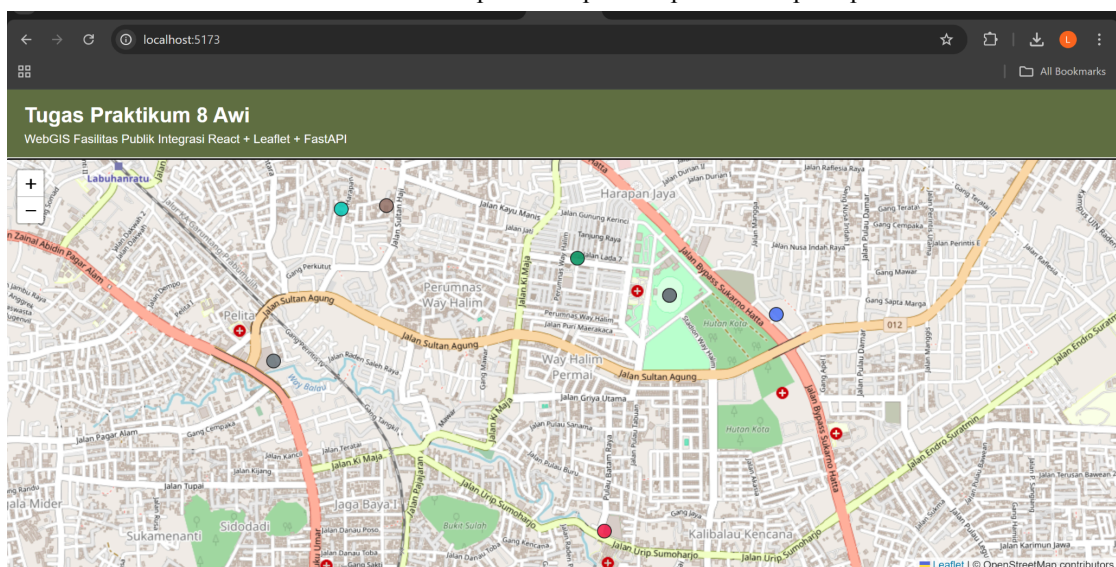
11. Menambahkan Popup Informasi

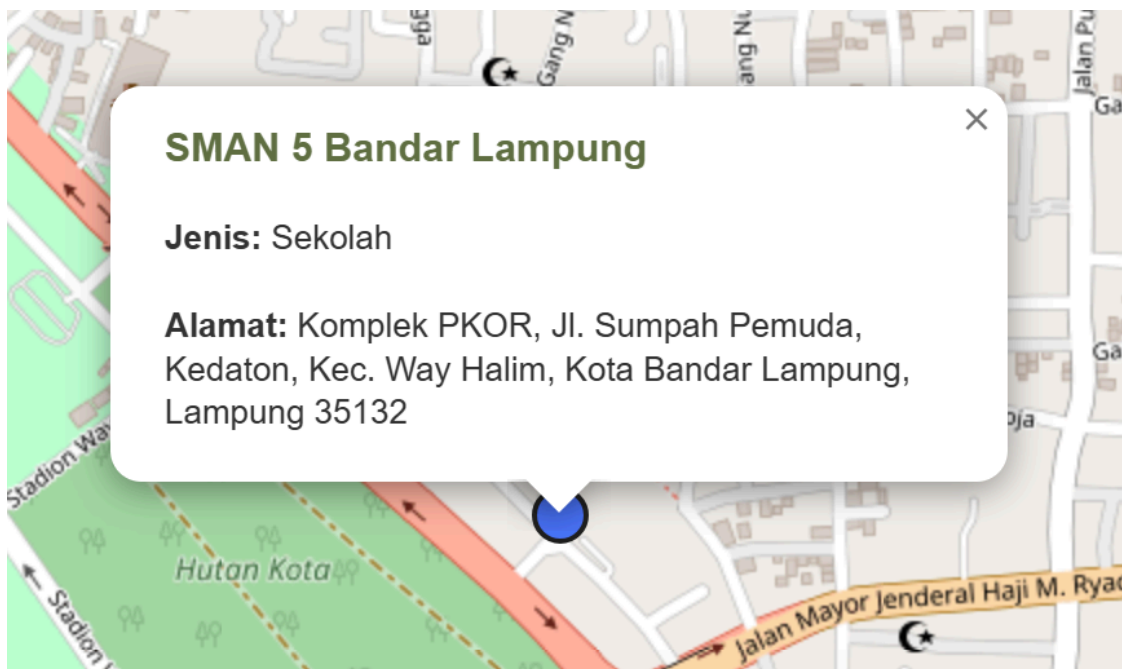
Agar terpenuhi syarat interaksi, saya menambahkan dua interaksi:

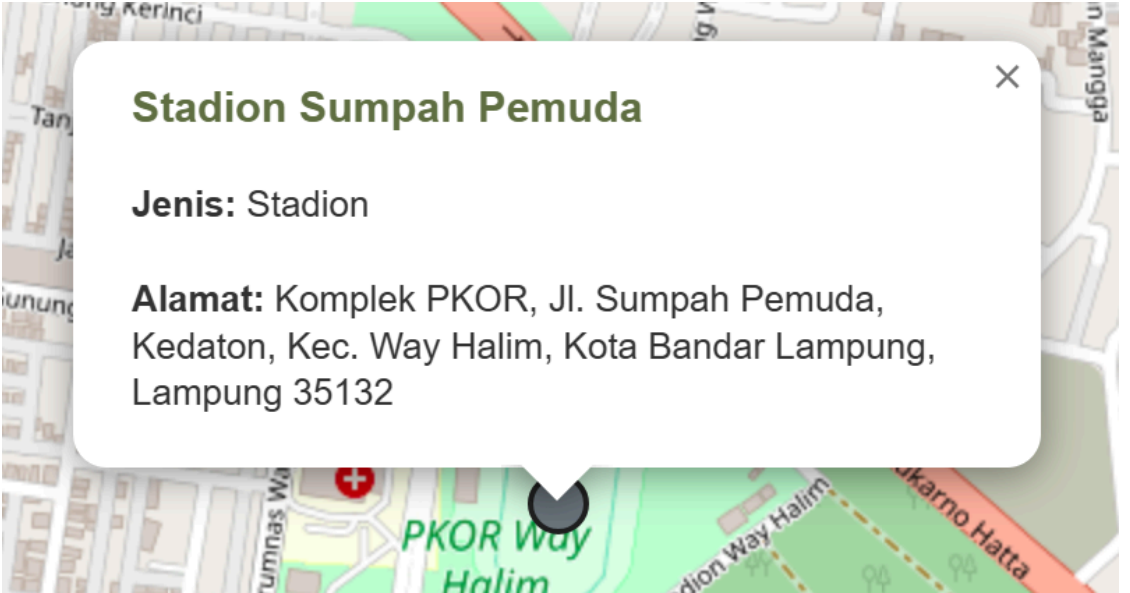
1. Hover highlight, saat mouse diarahkan ke titik, radius marker diperbesar dan tampil lebih menonjol.
2. Click zoom, saat titik diklik, peta akan melakukan `flyTo()` ke lokasi tersebut dengan zoom yang lebih dekat.

Interaksi ini ditambahkan melalui event handler di `layer.on({ ... })` pada `onEachFeature`. Dengan demikian, aplikasi tidak hanya menampilkan data, tetapi juga memberikan pengalaman interaktif kepada pengguna. Materi juga mencantumkan hover dan zoom to feature sebagai contoh interaksi yang bisa diterapkan.

Gambar 11. Hasil Kumpulan tampilan implementasi pada peta







Stadion Sumpah Pemuda

Jenis: Stadion

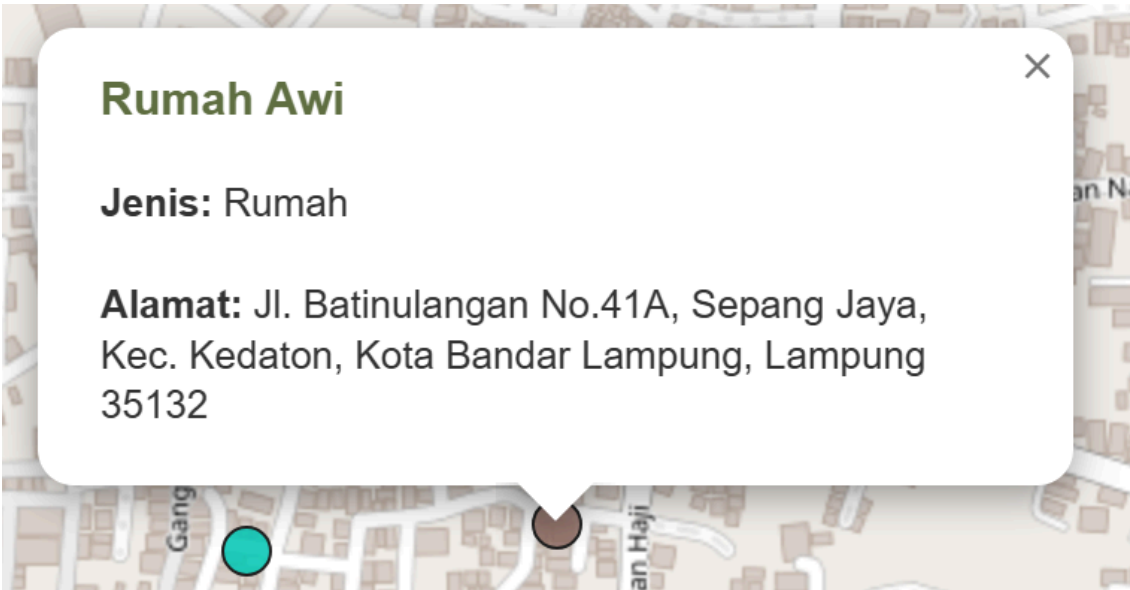
Alamat: Komplek PKOR, Jl. Sumpah Pemuda, Kedaton, Kec. Way Halim, Kota Bandar Lampung, Lampung 35132



Pasar Way Halim

Jenis: Pasar

Alamat: JL. Gunung Rajabasa Raya, No. T.21, Way Halim, 35361, Perumnas Way Halim, Kec. Way Halim, Kota Bandar Lampung, Lampung 35132



Rumah Awi

Jenis: Rumah

Alamat: Jl. Batinulangan No.41A, Sepang Jaya, Kec. Kedaton, Kota Bandar Lampung, Lampung 35132

